

TSFX: A Python package for time series feature extraction

Wilhelm Wermelin ^{1,2}

¹ RISE Research Institutes of Sweden ² Mälardalen University

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright¹³ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

TSFX is a Python ([Van Rossum & Drake, 2009](#)) library for extracting features from time series data. Time series feature extraction means deriving meaningful measures from a time series that are typically then used for downstream tasks such as visualization, classification, regression or similarity search. Mathematically, it is an operation that decouples the time series from the time index. Given a univariate time series of length T , X_t , where $t \in \{1, 2, \dots, T-1, T\}$ a feature extractor f is a mapping with the following characteristic:

$$f : X_t \mapsto \mathbb{R}$$

where $\mathbb{R}$ denotes the real numbers. An example of a time series feature is the mean (in which case the feature extractor is the mathematical operation of calculating the mean):

$$f_{\text{mean}}(X_t) = \bar{X} = \frac{1}{T} \sum_{t=0}^T X_t.$$

TSFX is a software library for calculating a multitude of such time series features with a focus on runtime performance.

Statement of need

Time series is a ubiquitous data modality, present in many domains such as finance, industry, meteorology, and medicine, to mention a few. As hardware to collect and store time series data is becoming increasingly affordable, the amount of available time series data is increasing in many domains.

A common preprocessing step when dealing with time series is feature extraction. This involves calculating representative features such as mean, variance, skewness, etc. from the time series to be used in downstream tasks such as classification, regression or clustering. For large time series datasets, performance is important for enabling timely data preprocessing. TSFX is made for this purpose: extracting features from large time series datasets.

Additionally, other more mature alternatives are based on Pandas ([McKinney, 2010](#)) DataFrames whereas TSFX is based on Polars ([Vink et al., 2023](#)) DataFrames. TSFX makes time series feature extraction readily available to developers/researchers using Polars in their projects.

State of the field

TSFX is inspired by the tsfresh library ([Christ et al., 2018](#)) and is to the author's knowledge the leading time series feature extraction library for Python (at least measured by GitHub

stars). Another popular time series feature extraction library is tsfel (Barandas et al., 2020) which offers similar functionality to the tsfresh.

As mentioned above, these libraries are based on Pandas DataFrames and while it is relatively easy to convert Pandas DataFrames to and from Polars equivalents, TSFX has access to Polars lazy execution, which can be a significant performance boost, being written specifically for Polars DataFrames (or LazyFrames).

Software design

TSFX puts an emphasis on performance on large time series datasets. To this end, it utilizes Polars (Vink et al., 2023) which is a fast DataFrame library written in Rust (The Rust Programming Language, 2015) with Python bindings facilitated through PyO3 (Project & Contributors, 2023). The feature extraction functions are implemented in Rust for fast execution. To benchmark, the “1 billion row challenge” (Morling, 2024) was used. In this benchmark, TSFX took approx. 250 seconds (using 64 CPU cores) to extract features vs tsfresh which took approx. 1500 seconds (using 80 CPU cores). Running repeated benchmarks, it has been observed that compared to tsfresh, TSFX offers about 5 to 10 times faster feature extraction, using the same set of time series features.

TSFX can be installed using pip:

```
pip install tsfx
```

TSFX can also be configured using a TOML (TOML, 2021) configuration file (default name .tsfx-config.toml).

Below is a simple example of extracting features from a time series dataset:

```
import polars as pl
from tsfx import (
    ExtractionSettings,
    FeatureSetting,
    extract_features,
)

df = pl.DataFrame(
    {
        "id": ["a", "a", "a", "b", "b", "b", "c", "c", "c"],
        "val": [1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 1.0, 2.0, 3.0],
        "value": [4.0, 5.0, 6.0, 6.0, 5.0, 4.0, 4.0, 5.0, 6.0],
    },
).lazy()
settings = ExtractionSettings(
    grouping_cols=["id"],
    feature_setting=FeatureSetting.Efficient,
    value_cols=["val", "value"],
)
gdf = extract_features(df, settings)
gdf = gdf.sort(by="id")
with pl.Config(set_tbl_width_chars=80):
    print(gdf)
```

Running the code above generates a new DataFrame with the extracted features:

```
shape: (3, 314)
```

id	length	val__su	val__me	...	value__	value__	value__	value__
a	3	1.0	2.0	3.0	4.0	5.0	6.0	...
b	3	1.0	2.0	3.0	4.0	5.0	6.0	...
c	3	1.0	2.0	3.0	4.0	5.0	6.0	...

---	---	m_value	an		number_	number_	number_	number_
str	u32	s	---		peaks__	peaks__	peaks__	peaks__
		f64	f64		n_3	n_5	n_10	n_50
					---	---	---	---
					f64	f64	f64	f64
a	3	6.0	2.0	...	0.0	0.0	0.0	0.0
b	3	6.0	2.0	...	0.0	0.0	0.0	0.0
c	3	6.0	2.0	...	0.0	0.0	0.0	0.0

53 If the DataFrame has a time column, it is also possible to extract over a time win-
54 dow by passing DynamicGroupBySettings into the feature extraction settings, like so:
55 ExtractionSettings(..., dynamic_settings=DynamicGroupBySettings(...)).

56 AI usage disclosure

57 Generative AI tools were used in the development of this software.

58 Google Gemini 3.1 Pro was used to help debugging and answer questions about the code.
59 Selected snippets generated by the tool have been incorporated into the software. No AI tool
60 added code independently to the codebase, it only provided snippets and suggestions.

61 The author have reviewed, edited, validated all AI-assisted outputs and made the core design
62 decisions. The authors take full responsibility for the accuracy, originality, licensing, and
63 ethical/legal compliance of all submitted materials.

64 Acknowledgements

65 The TSFX package was developed within the [Vinnova](#) projects [DFusion](#), [TolkAI](#), and [SIFT](#).
66 This research work has been funded by the Knowledge Foundation within the framework of
67 the INDTECH (Grant Number 20200132) and INDTECH+ Research School project (Grant
68 Number 20220132), participating companies and Mälardalen University.

69 References

- 70 Barandas, M., Folgado, D., Fernandes, L. cia, Santos, S., Abreu, M., Bota, P., Liu, H., Schultz,
71 T., & Gamboa, H. (2020). TSFEL: Time series feature extraction library. *SoftwareX*, 11,
72 100456.
- 73 Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series FeatuRe extrac-
74 tion on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*,
75 307, 72–77. <https://doi.org/https://doi.org/10.1016/j.neucom.2018.03.067>
- 76 McKinney, W. (2010). Data structures for statistical computing in python. In S. van der Walt
77 & J. Millman (Eds.), *Proceedings of the 9th python in science conference* (pp. 51–56).
- 78 Morling, G. (2024). *The one billion row challenge*. [https://www.morling.dev/blog/one-billion-](https://www.morling.dev/blog/one-billion-row-challenge/)
79 [row-challenge/](https://www.morling.dev/blog/one-billion-row-challenge/)
- 80 Project, P., & Contributors. (2023). *PyO3*. GitHub. <https://github.com/PyO3>
- 81 *The rust programming language*. (2015). <https://rust-lang.org/>.
- 82 *TOML: Tom's obvious minimal language*. (2021). <https://toml.io/en/>
- 83 Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace.
84 ISBN: 1441412697

85 Vink, R., Gooijer, S. de, Beedie, A., Zundert, J. van, Hulselmans, G., Grinstead, C., Gorelli, M.
86 E., Santamaria, M., Heres, D., ibENPC, Leitao, J., Heerden, M. van, Jermain, C., Russell,
87 R., Pryer, C., Castellanos, A. G., Goh, J., Wilksch, M., illumination-k, ... Keller, J. (2023).
88 *Pola-rs/polars: Python polars 0.16.11* (py-0.16.11). Zenodo. [https://doi.org/10.5281/](https://doi.org/10.5281/zenodo.7699984)
89 [zenodo.7699984](https://doi.org/10.5281/zenodo.7699984)

DRAFT